

Project 1 : Advanced Budget Tracker

1. Introduction :

Managing personal finances is an essential skill that affects everyone's quality of life. With increasing expenses and financial commitments, it becomes crucial to keep track of income, expenditures, and savings to maintain financial health and avoid debt.

The Advanced Budget Tracker is a simple yet powerful console-based application developed in Python. It helps users organize their monthly budgets by factoring in income and categorizing expenses with individual spending limits. The program generates insightful monthly reports, allowing users to understand their spending habits, manage money more responsibly, and plan for future savings.

This project is an excellent example of applying core programming concepts such as sequence, selection, iteration, and functions to solve real-world problems effectively.

2. Objectives :

The **Budget Tracker** is a simple Python program designed to help users track their incomes and expenses on a monthly basis. It allows you to:

- Add new income or expense entries
- View all transactions for a specific month
- View a summary of all months
- Clear all saved data

All data is saved in a plain text file named **budget-data.txt**.

3. Features Overview

The application incorporates several key features:

3.1 Add New Entry

Users can input the month, category, description, and amount for income (+) or expenses (-).

Example:

month: January 2025

category: Salary

description: Monthly salary

amount (+income/-expense): 5000

3.2 View Month Summary

Shows all entries for a specific month along with totals for income, expenses, and net balance.

Example Output:

```
--- January 2025 ---
Salary   | Monthly salary   | +5000.0
Rent     | Apartment rent   | -1200.0
-----
income: 5000.0 expense: 1200.0 balance: 3800.0
```

3.3 View All Months

Shows a summary of income, expenses, and balance for all months in the data file.

Example Output:

```
--- all months ---
January 2025 income: 5000.0 expense: 1200.0 balance: 3800.0
February 2025 income: 5000.0 expense: 400.0 balance: 4600.0
-----
```

3.4 Clear All Data

Deletes all saved data from budget-data.txt. Users are asked for confirmation before clearing.

3.5 Exit Program

Exits the budget tracker safely. Users are asked to confirm exit.

4. How the Program Works

- 4.1 `save_record(month_name, category_name, description_text, amount_value)`
Saves a single entry to the data file.
- 4.2 `read_records()`
Reads all entries from the data file and returns them as a list of tuples (`month_name`, `category_name`, `description_text`, `amount_value`).
- 4.3 `add_new_entry()`
Prompts the user to enter a new income/expense record and saves it.
- 4.4 `view_month()`
Prompts the user for a month and displays all entries and totals for that month.
- 4.5 `view_all()`
Displays the summary of all months with total income, expense, and balance.
- 4.6 `clear_all_data()`
Prompts the user to confirm and clears all data from the file.

5. Program

```
budget_file = "budget-data.txt"
```

```
def save_record(month_name, category_name, description_text, amount_value):
    file_handle = open(budget_file, "a")
    file_handle.write(month_name + "|" + category_name + "|" + description_text + "|" + str(amount_value)
+ "\n")
    file_handle.close()
#saves one record at a time
```

```
def read_records():
    all_records = []
    try:
        file_handle = open(budget_file, "r")
        for line_text in file_handle:
            parts = line_text.strip().split("|")
            if len(parts) == 4:
                month_name, category_name, description_text, amount_value = parts
                all_records.append((month_name, category_name, description_text, float(amount_value)))
        file_handle.close()
    except:
        pass
    return all_records
#it helps to read all records from the saved file
```

```
def add_new_entry():
    print("\nadding new entry")
    month_name = input("month: ")
    category_name = input("category: ")
    description_text = input("description: ")
    amount_value = float(input("amount (+income/-expense): "))
    save_record(month_name, category_name, description_text, amount_value)
    print("saved!\n")
#ask for input to add new entry
```

```
def view_month():
    month_to_view = input("month to view: ")
    all_records = read_records()
    month_entries = [entry for entry in all_records if entry[0] == month_to_view]
    if not month_entries:
```

```

    print("no records for that month\n")
    return
total_income = 0
total_expense = 0
print("\n---", month_to_view, "---")
for record in month_entries:
    month_name, category_name, description_text, amount_value = record
    sign_symbol = "+" if amount_value > 0 else "-"
    print(category_name + " | " + description_text + " | " + sign_symbol + str(abs(amount_value)))
    if amount_value > 0: total_income += amount_value
    else: total_expense += abs(amount_value)
balance_amount = total_income - total_expense
print("-----")
print("income:", total_income, "expense:", total_expense, "balance:", balance_amount, "\n")
#menu item to view month and details

def view_all():
    all_records = read_records()
    if not all_records:
        print("no data yet\n")
        return
    summary_by_month = {}
    for record in all_records:
        month_name, category_name, description_text, amount_value = record
        if month_name not in summary_by_month: summary_by_month[month_name] = {"income": 0,
"expense": 0}
        if amount_value > 0: summary_by_month[month_name]["income"] += amount_value
        else: summary_by_month[month_name]["expense"] += abs(amount_value)
    print("\n--- all months ---")
    for month_name, totals in summary_by_month.items():
        total_income = totals["income"]
        total_expense = totals["expense"]
        balance_amount = total_income - total_expense
        print(month_name, "income:", total_income, "expense:", total_expense, "balance:", balance_amount)
    print("-----\n")
#view every month income and expenses in a single list

def clear_all_data():
    confirm = input("delete all records? yes/no: ")
    if confirm.lower() == "yes":
        open(budget_file, "w").close()

```

```
    print("all cleared!\n")
else: print("cancelled\n")
#clear all saved data from the file

# main loop of the program that helps to add multiple entries!
while True:
    print("\n== welcome to the budget tracker made by Abhihi ==")
    print("1. add new entry")
    print("2. view a month")
    print("3. view all months")
    print("4. clear all data")
    print("5. exit")
    choice = input("option: ")
    if choice == "1": add_new_entry()
    elif choice == "2": view_month()
    elif choice == "3": view_all()
    elif choice == "4": clear_all_data()
    elif choice == "5":
        if input("really exit? y/n: ") == "y":
            print("goodbye!")
            break
    else:
        print("invalid choice\n") #wrong input by user
```

6. Output Example

```
== welcome to the budget tracker made by Abhihi ==
1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit
option: 1

adding new entry
month: jan 2025
category: Salary
description: credited
amount (+income/-expense): 2400
```

saved!

== welcome to the budget tracker made by Abhihi ==

1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit

option: 1

adding new entry

month: jan 2025

category: rent

description: paid

amount (+income/-expense): -1300

saved!

== welcome to the budget tracker made by Abhihi ==

1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit

option: 1

adding new entry

month: jan 2025

category: food

description: used

amount (+income/-expense): -100

saved!

== welcome to the budget tracker made by Abhihi ==

1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit

option: 1

adding new entry

month: feb 2025
category: salary
description: credited
amount (+income/-expense): 2400
saved!

== welcome to the budget tracker made by Abhiii ==

1. add new entry
 2. view a month
 3. view all months
 4. clear all data
 5. exit
- option: 1

adding new entry
month: rent
category: rent
description: paid
amount (+income/-expense): 1000
saved!

== welcome to the budget tracker made by Abhiii ==

1. add new entry
 2. view a month
 3. view all months
 4. clear all data
 5. exit
- option: 3

--- all months ---

jan 2025 income: 2400.0 expense: 1400.0 balance: 1000.0
feb 2025 income: 2400.0 expense: 0 balance: 2400.0
rent income: 1000.0 expense: 0 balance: 1000.0

== welcome to the budget tracker made by Abhiii ==

1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit

option: 4
delete all records? yes/no: yes
all cleared!

== welcome to the budget tracker made by Abhihi ==
1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit
option:

7. Problems Faced

While writing this budget tracker, I faced some problems with file handling and calculating total savings. Since the program saves all data in budget-data.txt, I had to be careful with reading and writing to the file, because any mistake or accidental deletion could cause loss of all records. Another problem I faced was figuring out how to calculate the total savings across multiple months. The program can show income, expenses, and balance for each month, but adding a running total for all months was tricky to implement, so I couldn't easily track overall savings while testing the code. I also faced some challenges with implementing more advanced features, so I had to take references from other code examples to understand how to structure the functions and manage the data efficiently.

8. What I can upgrade and Try

Total Savings Across Months

- Add a feature to calculate and display cumulative savings from all months.
- Example: after viewing all months, show Total Savings: \$8400.

Better Input Validation

- Check that the amount entered is a number.
- Ensure month names are consistent (maybe force the first letter uppercase).

Category Summary

- Show totals by category (e.g., total spent on Rent, Food, etc.) in a month.

Simple Sorting

- Display entries in order of amount (highest to lowest) or alphabetically by category.

Formatted Output

- Show amounts in a nice format with commas and two decimal places: \$1,200.00.

Basic Backup Option

- Create a simple copy of budget-data.txt before clearing data, so nothing is lost.

Add Notes or Tags

- Allow a short note for each entry to remember details like “paid online” or “cash purchase”.

Quick Month Selection

- Instead of typing the month name exactly, show a list of months with entries to choose from.

Simple Report

- After viewing all months, display the highest expense, highest income, and net balance.

9. References

- W3Schools
- [python.org](https://www.python.org)
- [learnpython.org](https://www.learnpython.org)
- Deepseek for project outline

10. Pictures

Codes

```

1 budget_file = "budget-data.txt"
2
3
4 def save_record(month_name, category_name, description_text, amount_value):
5     file_handle = open(budget_file, "a")
6     file_handle.write(month_name + "|" + category_name + "|" + description_text + "|" + str(amount_value) + "\n")
7     file_handle.close()
8     #saves one record at a time
9
10 def read_records():
11     all_records = []
12     try:
13         file_handle = open(budget_file, "r")
14         for line_text in file_handle:
15             parts = line_text.strip().split("|")
16             if len(parts) == 4:
17                 month_name, category_name, description_text, amount_value = parts
18                 all_records.append((month_name, category_name, description_text, float(amount_value)))
19         file_handle.close()
20     except:
21         pass
22     return all_records
23     #it helps to read all records from the saved file
24
25
26 def add_new_entry():
27     print("\nadding new entry")
28     month_name = input("month: ")
29     category_name = input("category: ")
30     description_text = input("description: ")
31     amount_value = float(input("amount (+income/-expense): "))
32     save_record(month_name, category_name, description_text, amount_value)
33     print("saved!\n")
34     #ask for input to add new entry
35
36
37 def view_month():
38     month_to_view = input("month to view: ")
39     all_records = read_records()
40     month_entries = [entry for entry in all_records if entry[0] == month_to_view]
41     if not month_entries:
42         print("no records for that month\n")
43         return
44     total_income = 0
45     total_expense = 0
46     print("\n---", month_to_view, "---")
47     for record in month_entries:
48         month_name, category_name, description_text, amount_value = record
49         sign_symbol = "+" if amount_value > 0 else "-"
50         print(category_name + " | " + description_text + " | " + sign_symbol + str(abs(amount_value)))
51         if amount_value > 0: total_income += amount_value
52         else: total_expense += abs(amount_value)
53     balance_amount = total_income - total_expense
54     print("-----")
55     print("income:", total_income, "expense:", total_expense, "balance:", balance_amount, "\n")
56     #menu item to view month and details
57
58
59 def view_all():
60     all_records = read_records()
61     if not all_records:
62         print("no data yet\n")
63         return
64     summary_by_month = {}
65     for record in all_records:
66         month_name, category_name, description_text, amount_value = record

```

```
66     month_name, category_name, description_text, amount_value = record
67     if month_name not in summary_by_month: summary_by_month[month_name] = {"income": 0, "expense": 0}
68     if amount_value > 0: summary_by_month[month_name]["income"] += amount_value
69     else: summary_by_month[month_name]["expense"] += abs(amount_value)
70     print("\n--- all months ---")
71     for month_name, totals in summary_by_month.items():
72         total_income = totals["income"]
73         total_expense = totals["expense"]
74         balance_amount = total_income - total_expense
75         print(month_name, "income:", total_income, "expense:", total_expense, "balance:", balance_amount)
76     print("-----\n")
77 #view every month income and expenses in a single list
78
79
80
81 def clear_all_data():
82     confirm = input("delete all records? yes/no: ")
83     if confirm.lower() == "yes":
84         open(budget_file, "w").close()
85         print("all cleared!\n")
86     else: print("cancelled\n")
87 #clear all saved data from the file
88
89
90 # main loop of the program that helps to add multiple entries!
91 while True:
92     print("\n== welcome to the budget tracker made by Abhiii ==")
93     print("1. add new entry")
94     print("2. view a month")
95     print("3. view all months")
96     print("4. clear all data")
97     print("5. exit")
98     choice = input("option: ")
99     if choice == "1": add_new_entry()
100    elif choice == "2": view_month()
101    elif choice == "3": view_all()
102    elif choice == "4": clear_all_data()
103    elif choice == "5":
104        if input("really exit? y/n: ") == "y":
105            print("goodbye!")
106            break
107    else:
108        print("invalid choice\n") #wrong input by user
109
```

Output

```
== welcome to the budget tracker made by Abhihi ==
1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit
option: 1

adding new entry
month: jan
category: salary
description: credited
amount (+income/-expense): 2400
saved!

== welcome to the budget tracker made by Abhihi ==
1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit
option: 1

adding new entry
month: jan
category: rent
description: paid
amount (+income/-expense): -1300
saved!

== welcome to the budget tracker made by Abhihi ==
1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit
option: 1

adding new entry
month: feb
category: salary
description: credited
amount (+income/-expense): 2400
saved!

== welcome to the budget tracker made by Abhihi ==
1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit
option: 1
```

```
== welcome to the budget tracker made by Abhiii ==
1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit
option: 1

adding new entry
month: feb
category: rent
description: 1100
amount (+income/-expense): -1100
saved!

== welcome to the budget tracker made by Abhiii ==
1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit
option: 2
month to view: jan

--- jan ---
salary | credited | +2400.0
rent | paid | -1300.0
-----
income: 2400.0 expense: 1300.0 balance: 1100.0

== welcome to the budget tracker made by Abhiii ==
1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit
option: 2
month to view: feb

--- feb ---
salary | credited | +2400.0
rent | 1100 | -1100.0
-----
income: 2400.0 expense: 1100.0 balance: 1300.0
```

```
--- feb ---
salary | credited | +2400.0
rent | 1100 | -1100.0
-----
income: 2400.0 expense: 1100.0 balance: 1300.0

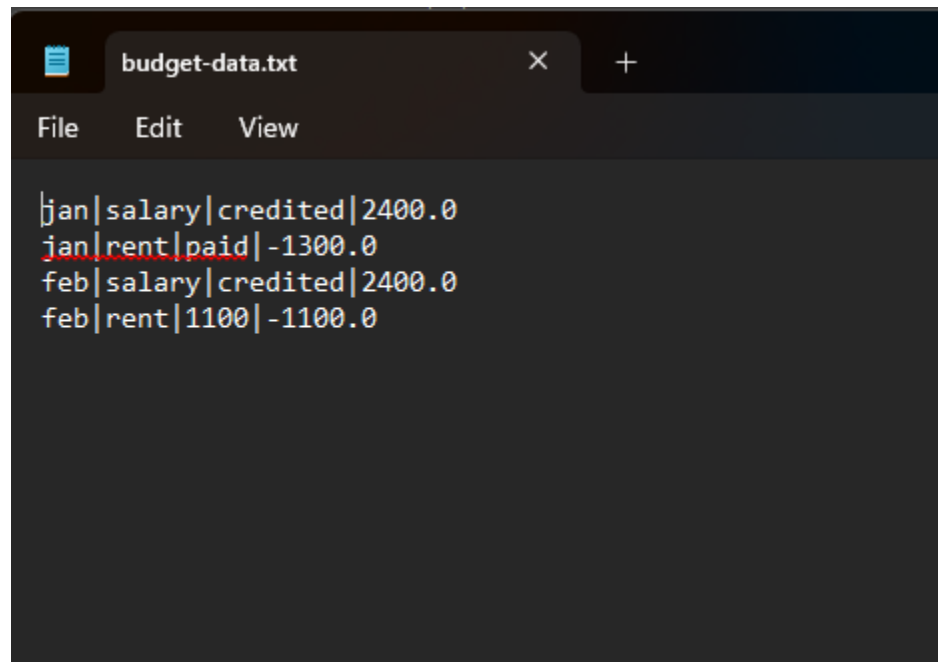
== welcome to the budget tracker made by Abhihi ==
1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit
option: 3

--- all months ---
jan income: 2400.0 expense: 1300.0 balance: 1100.0
feb income: 2400.0 expense: 1100.0 balance: 1300.0
-----

== welcome to the budget tracker made by Abhihi ==
1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit
option: 3

--- all months ---
jan income: 2400.0 expense: 1300.0 balance: 1100.0
feb income: 2400.0 expense: 1100.0 balance: 1300.0
-----

== welcome to the budget tracker made by Abhihi ==
1. add new entry
2. view a month
3. view all months
4. clear all data
5. exit
option:
```



```
budget-data.txt
File Edit View
jan|salary|credited|2400.0
jan|rent|paid|-1300.0
feb|salary|credited|2400.0
feb|rent|1100|-1100.0
```